

LAB ASSIGNMENT – 12.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- o Ask AI to include time complexity and space complexity in the function docstring.

- o Verify the generated code with test cases.

- Expected Output:

- o A functional Python script implementing Merge Sort with proper documentation.

Code:

```
# Generate a Python function merge_sort(arr) that sorts a list in ascending order using
the Merge Sort algorithm and include time complexity and space complexity in the
function docstring
```

```
def merge_sort(arr):
```

```
    """
```

```
    Sorts a list using Merge Sort algorithm.
```

```
    Time Complexity: O(n log n)
```

```
    Space Complexity: O(n)
```

```
    """
```

```
if len(arr) > 1:
```

```
    mid = len(arr) // 2
```

```
    left = arr[:mid]
```

```
    right = arr[mid:]
```

```
    merge_sort(left)
```

```
    merge_sort(right)
```

```
    i = j = k = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] < right[j]:
```

```
            arr[k] = left[i]
```

```
            i += 1
```

```
        else:
```

```
            arr[k] = right[j]
```

```
            j += 1
```

```
        k += 1
```

```
    while i < len(left):
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
        k += 1
```

```
    while j < len(right):
```

```
        arr[k] = right[j]
```

```
        j += 1
```

```
k += 1
```

```
return arr
```

Example usage

```
arr = [38, 27, 43, 3, 9, 82, 10]
```

```
print("Sorted Array:", merge_sort(arr))
```

Output:

```
C:\Users\boora> Downloads\3-2\AI-AC> acc125.py -
1  # Generate a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm and include time complexity and space complexity in the function docstri
2  def merge_sort(arr):
3      """
4      Sorts a list using Merge Sort algorithm.
5
6      Time Complexity: O(n log n)
7      Space Complexity: O(n)
8      """
9
10     if len(arr) > 1:
11         mid = len(arr) // 2
12         left = arr[:mid]
13         right = arr[mid:]
14
15         merge_sort(left)
16         merge_sort(right)
17
18         i = j = k = 0
19
20         while i < len(left) and j < len(right):
21             if left[i] < right[j]:
22                 arr[k] = left[i]
23                 i += 1
24             else:
25                 arr[k] = right[j]
26                 j += 1
27             k += 1
28
29         while i < len(left):
30             arr[k] = left[i]
31             i += 1
32             k += 1
33
34         while j < len(right):
35             arr[k] = right[j]
36             j += 1
37             k += 1
38
39     return arr
40
41 if __name__ == '__main__':
42     arr = [38, 27, 43, 3, 9, 82, 10]
43     sorted_arr = merge_sort(arr)
44     print("Sorted Array:", sorted_arr)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\boora\Downloads\3-2\AI-AC> & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\
libs\debugpy\launcher' '62486' '-.' 'C:\Users\boora\Downloads\3-2\AI-AC\acc12.5.py'
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

Merge Sort is an efficient divide-and-conquer sorting algorithm. It splits the list into smaller sublists and merges them in sorted order. It guarantees $O(n \log n)$ time complexity and works well for large datasets.

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.

- Expected Output:

- o Python code implementing binary search with AI-generated comments and docstrings.

Code:

```
# "Generate a Python function binary_search(arr, target) that searches for a target element in a sorted list and returns its index or -1 if not found, including docstrings explaining best, average, and worst-case time complexities
```

```
def binary_search(arr, target):
```

```
    """
```

```
    Performs Binary Search on a sorted list.
```

```
    Best Case Complexity: O(1) -> when target is found at the middle.
```

```
    Average Case Complexity: O(log n)
```

```
    Worst Case Complexity: O(log n)
```

```
    Returns:
```

```
    Index of target if found, otherwise -1.
```

```
"""
```

```
left = 0
```

```
right = len(arr) - 1
```

```
while left <= right:
```

```
    mid = (left + right) // 2
```

```
    if arr[mid] == target:
```

```
        return mid
```

```
    elif arr[mid] < target:
```

```
        left = mid + 1
```

```
    else:
```

```
        right = mid - 1
```

```
return -1
```

```
# Test Cases
```

```
arr = [2, 5, 8, 12, 16, 23, 38]
```

```
print(binary_search(arr, 12)) # Output: 3
```

```
print(binary_search(arr, 5)) # Output: 1
```

```
print(binary_search(arr, 20)) # Output: -1
```

```
Output:
```


Code:

```
# Here is the one-line prompt for Task Description #3:
```

```
#"Generate a Python program for a Smart Healthcare Appointment Scheduling System that stores appointment ID, patient name, doctor name, appointment time, and consultation fee, uses Binary Search to find appointments by ID, and uses Merge Sort to sort appointments by time or consultation fee with proper comments and explanations."**
```

```
# Smart Healthcare Appointment Scheduling System
```

```
# Appointment records
```

```
appointments = [  
    {"id":101, "patient":"Ravi", "doctor":"Dr.Kumar", "time":10, "fee":500},  
    {"id":102, "patient":"Anita", "doctor":"Dr.Rao", "time":12, "fee":300},  
    {"id":103, "patient":"Sita", "doctor":"Dr.Mehta", "time":9, "fee":700},  
    {"id":104, "patient":"Rahul", "doctor":"Dr.Sharma", "time":11, "fee":400}  
]
```

```
# Binary Search to find appointment by ID
```

```
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["id"] < target:
```

```
        left = mid + 1
    else:
        right = mid - 1

    return "Appointment not found"
```

Merge Sort to sort appointments

```
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
```



```
result.extend(right[j:])
```

```
return result
```

```
# Sort appointments by time
```

```
sorted_time = merge_sort(appointments, "time")
```

```
print("Sorted by Time:", sorted_time)
```

```
# Sort appointments by consultation fee
```

```
sorted_fee = merge_sort(appointments, "fee")
```

```
print("Sorted by Fee:", sorted_fee)
```

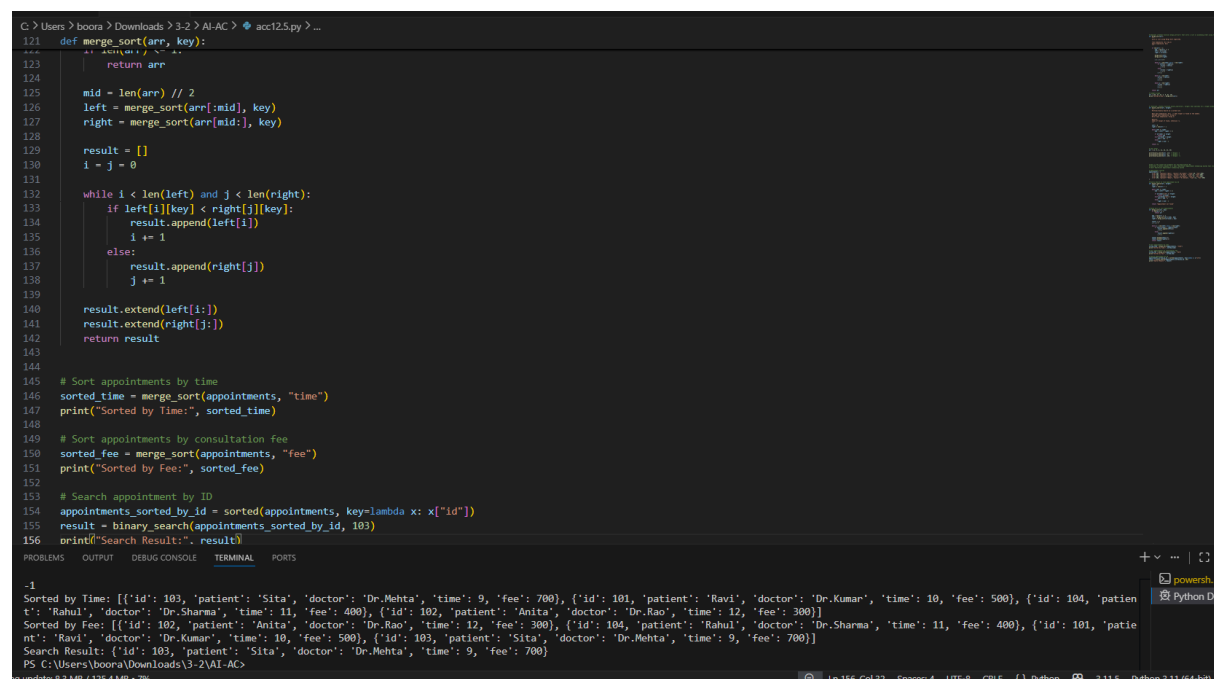
```
# Search appointment by ID
```

```
appointments_sorted_by_id = sorted(appointments, key=lambda x: x["id"])
```

```
result = binary_search(appointments_sorted_by_id, 103)
```

```
print("Search Result:", result)
```

Output:



The screenshot shows a VS Code editor with a Python script in a file named `acc12.5.py`. The script defines a `merge_sort` function and uses it to sort a list of appointments by time, fee, and ID. It also performs a binary search for an appointment with ID 103. The terminal output shows the results of these operations.

```
C:\Users\boora > Downloads > 3-2 > AI-AC > acc12.5.py > ...
121 def merge_sort(arr, key):
122     return arr
123
124
125     mid = len(arr) // 2
126     left = merge_sort(arr[:mid], key)
127     right = merge_sort(arr[mid:], key)
128
129     result = []
130     i = j = 0
131
132     while i < len(left) and j < len(right):
133         if left[i][key] < right[j][key]:
134             result.append(left[i])
135             i += 1
136         else:
137             result.append(right[j])
138             j += 1
139
140     result.extend(left[i:])
141     result.extend(right[j:])
142     return result
143
144
145 # Sort appointments by time
146 sorted_time = merge_sort(appointments, "time")
147 print("Sorted by Time:", sorted_time)
148
149 # Sort appointments by consultation fee
150 sorted_fee = merge_sort(appointments, "fee")
151 print("Sorted by Fee:", sorted_fee)
152
153 # Search appointment by ID
154 appointments_sorted_by_id = sorted(appointments, key=lambda x: x["id"])
155 result = binary_search(appointments_sorted_by_id, 103)
156 print("Search Result:", result)
```

Terminal Output:

```
-1
Sorted by Time: [{'id': 103, 'patient': 'Sita', 'doctor': 'Dr.Mehta', 'time': 9, 'fee': 700}, {'id': 101, 'patient': 'Ravi', 'doctor': 'Dr.Kumar', 'time': 10, 'fee': 500}, {'id': 104, 'patient': 'Rahul', 'doctor': 'Dr.Sharma', 'time': 11, 'fee': 400}, {'id': 102, 'patient': 'Anita', 'doctor': 'Dr.Rao', 'time': 12, 'fee': 300}]
Sorted by Fee: [{'id': 102, 'patient': 'Anita', 'doctor': 'Dr.Rao', 'time': 12, 'fee': 300}, {'id': 104, 'patient': 'Rahul', 'doctor': 'Dr.Sharma', 'time': 11, 'fee': 400}, {'id': 101, 'patient': 'Ravi', 'doctor': 'Dr.Kumar', 'time': 10, 'fee': 500}, {'id': 103, 'patient': 'Sita', 'doctor': 'Dr.Mehta', 'time': 9, 'fee': 700}]
Search Result: {'id': 103, 'patient': 'Sita', 'doctor': 'Dr.Mehta', 'time': 9, 'fee': 700}
```

Justification:

Binary Search is used to quickly find appointments using *appointment ID* in sorted records. Merge Sort efficiently organizes appointments based on *time or consultation fee*. These algorithms improve system performance when handling large appointment datasets.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Code:

```
# """Generate a Python program for a Railway Ticket Reservation System that stores
ticket ID, passenger name, train number, seat number, and travel date, uses Binary
Search to find tickets by ticket ID, and uses Merge Sort to sort bookings by travel date or
seat number with proper comments and documentation."""
```

```
# Railway Ticket Records
```

```
tickets = [
    {"id":201, "passenger":"Ramesh", "train":1201, "seat":15, "date":"2026-04-01"},
    {"id":202, "passenger":"Sita", "train":1205, "seat":10, "date":"2026-03-28"},
```

```
{ "id":203, "passenger":"Rahul", "train":1210, "seat":8, "date":"2026-04-05"},  
{ "id":204, "passenger":"Anita", "train":1199, "seat":20, "date":"2026-03-30"}  
]
```

```
# Binary Search for Ticket ID
```

```
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["id"] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return "Ticket not found"
```

```
# Merge Sort for sorting tickets
```

```
def merge_sort(arr, key):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
left = merge_sort(arr[:mid], key)
right = merge_sort(arr[mid:], key)
```

```
result = []
```

```
i = j = 0
```

```
while i < len(left) and j < len(right):
```

```
    if left[i][key] < right[j][key]:
```

```
        result.append(left[i])
```

```
        i += 1
```

```
    else:
```

```
        result.append(right[j])
```

```
        j += 1
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
# Sort by travel date
```

```
sorted_date = merge_sort(tickets, "date")
```

```
print("Sorted by Travel Date:", sorted_date)
```

```
# Sort by seat number
```

```
sorted_seat = merge_sort(tickets, "seat")
```

```
print("Sorted by Seat Number:", sorted_seat)
```

```
# Search ticket by ID
```

```

tickets_sorted_by_id = sorted(tickets, key=lambda x: x["id"])

print("Search Result:", binary_search(tickets_sorted_by_id, 203))

```

Output:

```

C:\Users\boora>boora>Downloads>3-2>AI-AC> acc125.py >...
160
161 # **Generate a Python program for a Railway Ticket Reservation System that stores ticket ID, passenger name, train number, seat number, and travel date, uses Binary Search to find ticket details.
162 # Railway Ticket Records
163 tickets = [
164     {"id":201, "passenger":"Ramesh", "train":1201, "seat":15, "date":"2026-04-01"},
165     {"id":202, "passenger":"Sita", "train":1205, "seat":10, "date":"2026-03-28"},
166     {"id":203, "passenger":"Rahul", "train":1210, "seat":8, "date":"2026-04-05"},
167     {"id":204, "passenger":"Anita", "train":1199, "seat":20, "date":"2026-03-30"}
168 ]
169
170 # Binary Search for Ticket ID
171 def binary_search(arr, target):
172     left = 0
173     right = len(arr) - 1
174
175     while left <= right:
176         mid = (left + right) // 2
177
178         if arr[mid]["id"] == target:
179             return arr[mid]
180         elif arr[mid]["id"] < target:
181             left = mid + 1
182         else:
183             right = mid - 1
184
185     return "Ticket not found"
186
187
188 # Merge Sort for sorting tickets
189 def merge_sort(arr, key):
190     if len(arr) <= 1:
191         return arr
192
193     mid = len(arr) // 2
194     left = arr[:mid]
195     right = arr[mid:]
196
197     left = merge_sort(left, key)
198     right = merge_sort(right, key)
199
200     return merge(left, right, key)
201
202 def merge(left, right, key):
203     result = []
204     while len(left) > 0 and len(right) > 0:
205         if left[0][key] < right[0][key]:
206             result.append(left.pop(0))
207         else:
208             result.append(right.pop(0))
209     result.extend(left)
210     result.extend(right)
211     return result
212
213 # Test the functions
214 # Sort by Travel Date
215 sorted_by_date = merge_sort(tickets, "date")
216 print("Sorted by Travel Date:", sorted_by_date)
217
218 # Sort by Seat Number
219 sorted_by_seat = merge_sort(tickets, "seat")
220 print("Sorted by Seat Number:", sorted_by_seat)
221
222 # Search for a ticket
223 search_result = binary_search(tickets, 203)
224 print("Search Result:", search_result)

```

Search Result: {'id': 203, 'passenger': 'Rahul', 'train': 1210, 'seat': 8, 'date': '2026-04-05'}

Sorted by Travel Date: [{'id': 202, 'passenger': 'Sita', 'train': 1205, 'seat': 10, 'date': '2026-03-28'}, {'id': 204, 'passenger': 'Anita', 'train': 1199, 'seat': 20, 'date': '2026-03-30'}, {'id': 201, 'passenger': 'Ramesh', 'train': 1201, 'seat': 15, 'date': '2026-04-01'}, {'id': 203, 'passenger': 'Rahul', 'train': 1210, 'seat': 8, 'date': '2026-04-05'}]

Sorted by Seat Number: [{'id': 203, 'passenger': 'Rahul', 'train': 1210, 'seat': 8, 'date': '2026-04-05'}, {'id': 202, 'passenger': 'Sita', 'train': 1205, 'seat': 10, 'date': '2026-03-28'}, {'id': 201, 'passenger': 'Ramesh', 'train': 1201, 'seat': 15, 'date': '2026-04-01'}, {'id': 204, 'passenger': 'Anita', 'train': 1199, 'seat': 20, 'date': '2026-03-30'}]

Search Result: {'id': 203, 'passenger': 'Rahul', 'train': 1210, 'seat': 8, 'date': '2026-04-05'}

PS C:\Users\boora\Downloads\3-2\AI-AC>

Justification:

Binary Search helps quickly locate ticket details using *ticket ID. Merge Sort is used to arrange booking records by **travel date or seat number*. These algorithms ensure efficient searching and sorting in railway reservation systems.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Code:

```
# Generate a Python program for a Smart Hostel Room Allocation System that stores
student ID, room number, floor, and allocation date, uses Binary Search to find records
by student ID, and uses Merge Sort to sort records by room number or allocation date
with proper comments
```

```
# Hostel room allocation records
```

```
allocations = [
    {"student_id":101, "room":205, "floor":2, "date":"2026-03-01"},
    {"student_id":102, "room":101, "floor":1, "date":"2026-02-25"},
    {"student_id":103, "room":310, "floor":3, "date":"2026-03-05"},
    {"student_id":104, "room":150, "floor":1, "date":"2026-02-28"}
]
```

```
# Binary Search for Student ID
```

```
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["student_id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["student_id"] < target:
```

```
        left = mid + 1
    else:
        right = mid - 1

    return "Student allocation not found"
```

Merge Sort for sorting records

```
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
# Sort by room number
```

```
sorted_room = merge_sort(allocations, "room")
```

```
print("Sorted by Room Number:", sorted_room)
```

```
# Sort by allocation date
```

```
sorted_date = merge_sort(allocations, "date")
```

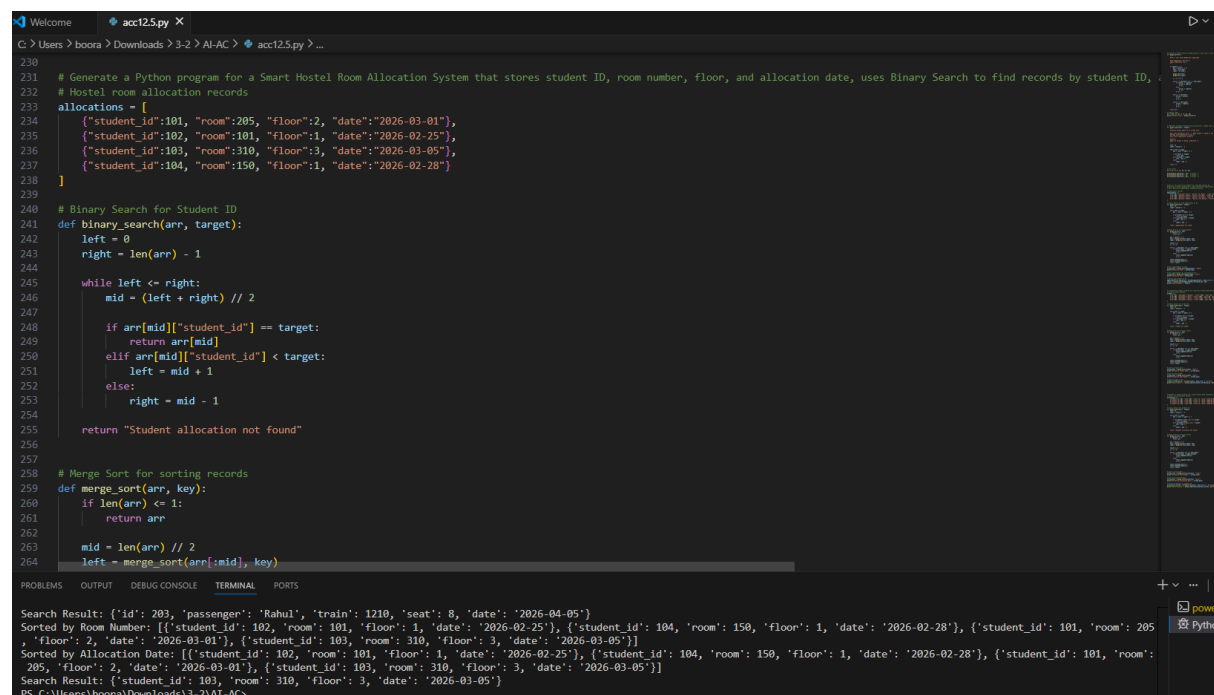
```
print("Sorted by Allocation Date:", sorted_date)
```

```
# Search allocation by student ID
```

```
allocations_sorted = sorted(allocations, key=lambda x: x["student_id"])
```

```
print("Search Result:", binary_search(allocations_sorted, 103))
```

Output:



```
230
231 # Generate a Python program for a Smart Hostel Room Allocation System that stores student ID, room number, floor, and allocation date, uses Binary Search to find records by student ID,
232 # Hostel room allocation records
233 allocations = [
234     {"student_id":101, "room":205, "floor":2, "date":"2026-03-01"},
235     {"student_id":102, "room":101, "floor":1, "date":"2026-02-25"},
236     {"student_id":103, "room":310, "floor":3, "date":"2026-03-05"},
237     {"student_id":104, "room":150, "floor":1, "date":"2026-02-28"}
238 ]
239
240 # Binary Search for Student ID
241 def binary_search(arr, target):
242     left = 0
243     right = len(arr) - 1
244
245     while left <= right:
246         mid = (left + right) // 2
247
248         if arr[mid]["student_id"] == target:
249             return arr[mid]
250         elif arr[mid]["student_id"] < target:
251             left = mid + 1
252         else:
253             right = mid - 1
254
255     return "Student allocation not found"
256
257
258 # Merge Sort for sorting records
259 def merge_sort(arr, key):
260     if len(arr) <= 1:
261         return arr
262
263     mid = len(arr) // 2
264     left = merge_sort(arr[:mid], key)
```

Search Result: {'id': 203, 'passenger': 'Rahul', 'train': 1210, 'seat': 8, 'date': '2026-04-05'}

Sorted by Room Number: [{'student_id': 102, 'room': 101, 'floor': 1, 'date': '2026-02-25'}, {'student_id': 104, 'room': 150, 'floor': 1, 'date': '2026-02-28'}, {'student_id': 101, 'room': 205, 'floor': 2, 'date': '2026-03-01'}, {'student_id': 103, 'room': 310, 'floor': 3, 'date': '2026-03-05'}]

Sorted by Allocation Date: [{'student_id': 102, 'room': 101, 'floor': 1, 'date': '2026-02-25'}, {'student_id': 104, 'room': 150, 'floor': 1, 'date': '2026-02-28'}, {'student_id': 101, 'room': 205, 'floor': 2, 'date': '2026-03-01'}, {'student_id': 103, 'room': 310, 'floor': 3, 'date': '2026-03-05'}]

Search Result: {'student_id': 103, 'room': 310, 'floor': 3, 'date': '2026-03-05'}

PS C:\Users\boora\Downloads\3-2> AT-AC

Justification:

Binary Search efficiently finds student allocation details using *student ID. Merge Sort organizes records by **room number or allocation date*. This improves data management and retrieval in hostel systems.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

Code:

```
# Generate a Python program for an Online Movie Streaming Platform that stores movie ID, title, genre, rating, and release year, uses Binary Search to find movies by movie ID, and uses Merge Sort to sort movies by rating or release year with proper comments
```

```
# Movie records
```

```
movies = [  
    {"id":1, "title":"Inception", "genre":"Sci-Fi", "rating":8.8, "year":2010},  
    {"id":2, "title":"Avatar", "genre":"Action", "rating":7.8, "year":2009},  
    {"id":3, "title":"Interstellar", "genre":"Sci-Fi", "rating":8.6, "year":2014},  
    {"id":4, "title":"Titanic", "genre":"Romance", "rating":7.9, "year":1997}  
]
```

```
# Binary Search by Movie ID
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["id"] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return "Movie not found"
```

```
# Merge Sort for sorting movies
```

```
def merge_sort(arr, key):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid], key)
```

```
    right = merge_sort(arr[mid:], key)
```

```
result = []  
  
i = j = 0  
  
while i < len(left) and j < len(right):  
    if left[i][key] < right[j][key]:  
        result.append(left[i])  
        i += 1  
    else:  
        result.append(right[j])  
        j += 1
```

```
result.extend(left[i:])  
result.extend(right[j:])  
return result
```

Sort by rating

```
sorted_rating = merge_sort(movies, "rating")  
print("Sorted by Rating:", sorted_rating)
```

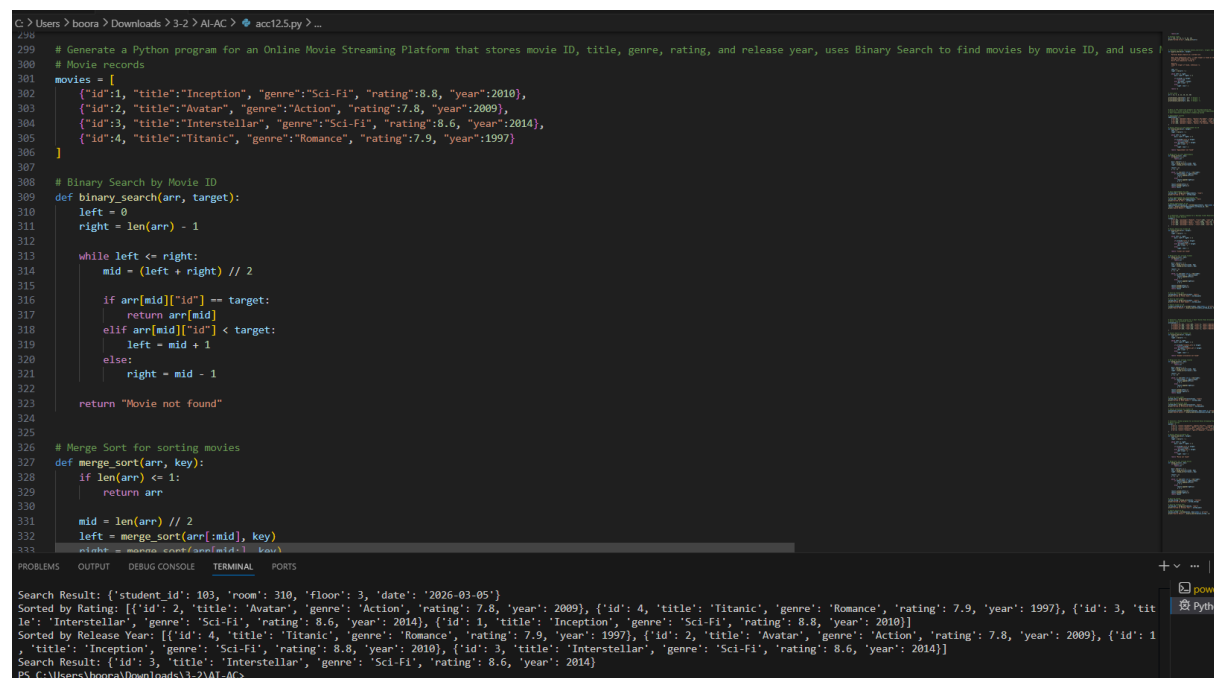
Sort by release year

```
sorted_year = merge_sort(movies, "year")  
print("Sorted by Release Year:", sorted_year)
```

Search movie by ID

```
movies_sorted = sorted(movies, key=lambda x: x["id"])  
print("Search Result:", binary_search(movies_sorted, 3))
```

Output:



```
G:\Users\boora\Downloads\3-2> AI-AC > acc125.py ...
299 # Generate a Python program for an Online Movie Streaming Platform that stores movie ID, title, genre, rating, and release year, uses Binary Search to find movies by movie ID, and uses
300 # Movie records
301 movies = [
302     {"id":1, "title":"Inception", "genre":"Sci-Fi", "rating":8.8, "year":2010},
303     {"id":2, "title":"Avatar", "genre":"Action", "rating":7.8, "year":2009},
304     {"id":3, "title":"Interstellar", "genre":"Sci-Fi", "rating":8.6, "year":2014},
305     {"id":4, "title":"Titanic", "genre":"Romance", "rating":7.9, "year":1997}
306 ]
307
308 # Binary Search by Movie ID
309 def binary_search(arr, target):
310     left = 0
311     right = len(arr) - 1
312
313     while left <= right:
314         mid = (left + right) // 2
315
316         if arr[mid]["id"] == target:
317             return arr[mid]
318         elif arr[mid]["id"] < target:
319             left = mid + 1
320         else:
321             right = mid - 1
322
323     return "Movie not found"
324
325
326 # Merge Sort for sorting movies
327 def merge_sort(arr, key):
328     if len(arr) <= 1:
329         return arr
330
331     mid = len(arr) // 2
332     left = merge_sort(arr[:mid], key)
333     right = merge_sort(arr[mid:], key)
334
335     return merge(left, right, key)
336
337 def merge(left, right, key):
338     result = []
339     while left and right:
340         if left[0][key] < right[0][key]:
341             result.append(left.pop(0))
342         else:
343             result.append(right.pop(0))
344     result.extend(left)
345     result.extend(right)
346     return result
347
348 # Search for a movie
349 target_id = 3
350 result = binary_search(movies, target_id)
351 print("Search Result:", result)
352
353 # Sort movies by rating
354 sorted_movies = merge_sort(movies, "rating")
355 print("Sorted by Rating:", sorted_movies)
356
357 # Sort movies by release year
358 sorted_movies = merge_sort(movies, "year")
359 print("Sorted by Release Year:", sorted_movies)
```

Search Result: {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}

Sorted by Rating: [{'id': 2, 'title': 'Avatar', 'genre': 'Action', 'rating': 7.8, 'year': 2009}, {'id': 4, 'title': 'Titanic', 'genre': 'Romance', 'rating': 7.9, 'year': 1997}, {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}, {'id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}]

Sorted by Release Year: [{'id': 4, 'title': 'Titanic', 'genre': 'Romance', 'rating': 7.9, 'year': 1997}, {'id': 2, 'title': 'Avatar', 'genre': 'Action', 'rating': 7.8, 'year': 2009}, {'id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}, {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}]

Search Result: {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}

PS C:\Users\boora\Downloads\3-2> AI-AC >

Justification:

Binary Search helps quickly search movie records using *movie ID. Merge Sort is suitable for arranging movies by **rating or release year*. These algorithms allow efficient handling of large movie databases.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.

- Justify algorithm suitability.
- Implement searching and sorting in Python.

Code:

```
# Generate a Python program for a Smart Agriculture Crop Monitoring System that
stores crop ID, crop name, soil moisture level, temperature, and yield estimate, uses
Binary Search to find crops by crop ID, and uses Merge Sort to sort crops by moisture
level or yield estimate with proper comments
```

```
# Crop data records
```

```
crops = [
    {"id":1, "name":"Wheat", "moisture":30, "temperature":25, "yield":200},
    {"id":2, "name":"Rice", "moisture":45, "temperature":28, "yield":350},
    {"id":3, "name":"Corn", "moisture":35, "temperature":27, "yield":300},
    {"id":4, "name":"Barley", "moisture":25, "temperature":22, "yield":180}
]
```

```
# Binary Search by Crop ID
```

```
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["id"] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
right = mid - 1
```

```
return "Crop not found"
```

```
# Merge Sort for sorting crops
```

```
def merge_sort(arr, key):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid], key)
```

```
    right = merge_sort(arr[mid:], key)
```

```
    result = []
```

```
    i = j = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i][key] < right[j][key]:
```

```
            result.append(left[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right[j])
```

```
            j += 1
```

```
    result.extend(left[i:])
```

```
    result.extend(right[j:])
```

```
    return result
```

```
# Sort crops by moisture level
```

```
sorted_moisture = merge_sort(crops, "moisture")
```

```
print("Sorted by Moisture:", sorted_moisture)
```

```
# Sort crops by yield estimate
```

```
sorted_yield = merge_sort(crops, "yield")
```

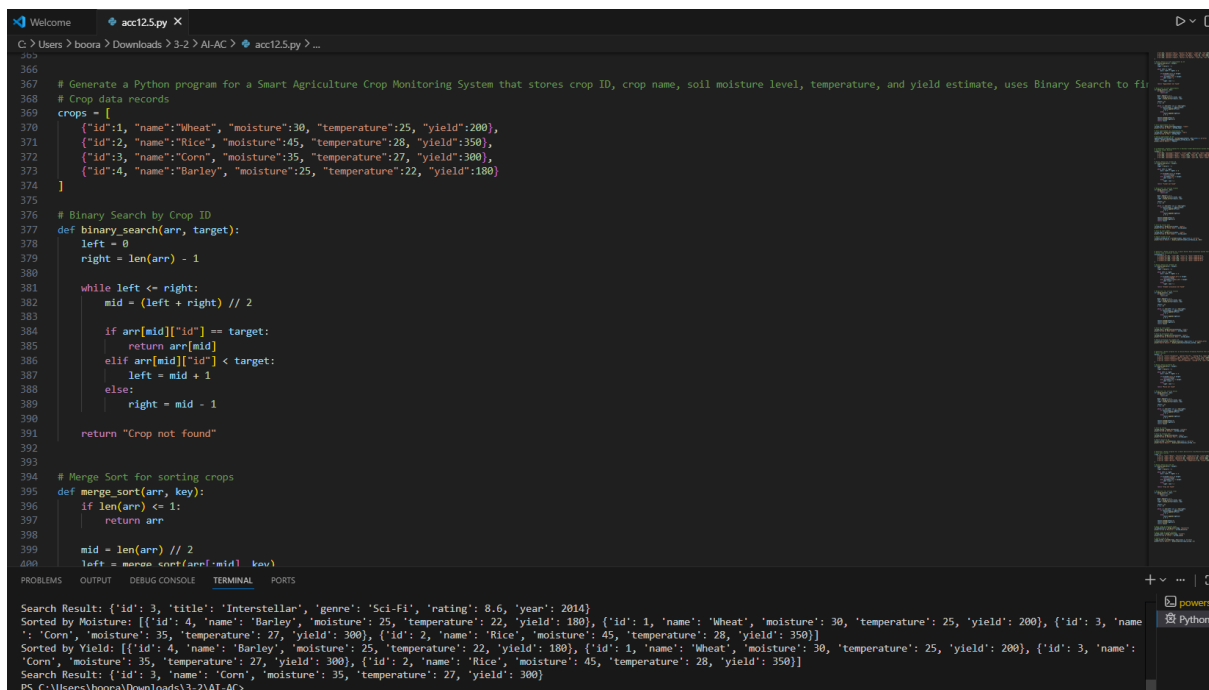
```
print("Sorted by Yield:", sorted_yield)
```

```
# Search crop by ID
```

```
crops_sorted = sorted(crops, key=lambda x: x["id"])
```

```
print("Search Result:", binary_search(crops_sorted, 3))
```

Output:



The screenshot shows a VS Code editor with a Python file named `acc125.py`. The code defines a list of crops, a binary search function, and a merge sort function. The terminal output shows the results of these functions.

```
366
367 # Generate a Python program for a Smart Agriculture Crop Monitoring System that stores crop ID, crop name, soil moisture level, temperature, and yield estimate, uses Binary Search to find
368 # Crop data records
369 crops = [
370     {"id":1, "name":"Wheat", "moisture":30, "temperature":25, "yield":200},
371     {"id":2, "name":"Rice", "moisture":45, "temperature":28, "yield":350},
372     {"id":3, "name":"Corn", "moisture":35, "temperature":27, "yield":300},
373     {"id":4, "name":"Barley", "moisture":25, "temperature":22, "yield":180}
374 ]
375
376 # Binary Search by Crop ID
377 def binary_search(arr, target):
378     left = 0
379     right = len(arr) - 1
380
381     while left <= right:
382         mid = (left + right) // 2
383
384         if arr[mid]["id"] == target:
385             return arr[mid]
386         elif arr[mid]["id"] < target:
387             left = mid + 1
388         else:
389             right = mid - 1
390
391     return "Crop not found"
392
393
394 # Merge Sort for sorting crops
395 def merge_sort(arr, key):
396     if len(arr) <= 1:
397         return arr
398
399     mid = len(arr) // 2
400     left = merge_sort(arr[:mid], key)
401     right = merge_sort(arr[mid:], key)
402
403     return merge(left, right, key)
404
405 def merge(left, right, key):
406     result = []
407     while left and right:
408         if left[0][key] < right[0][key]:
409             result.append(left.pop(0))
410         else:
411             result.append(right.pop(0))
412     result.extend(left)
413     result.extend(right)
414     return result
415
416 # Test the functions
417 crops_sorted = sorted(crops, key=lambda x: x["id"])
418 search_result = binary_search(crops_sorted, 3)
419 sorted_moisture = merge_sort(crops, "moisture")
420 sorted_yield = merge_sort(crops, "yield")
421
422 print("Search Result:", search_result)
423 print("Sorted by Moisture:", sorted_moisture)
424 print("Sorted by Yield:", sorted_yield)
```

Search Result: {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}

Sorted by Moisture: [{'id': 4, 'name': 'Barley', 'moisture': 25, 'temperature': 22, 'yield': 180}, {'id': 1, 'name': 'Wheat', 'moisture': 30, 'temperature': 25, 'yield': 200}, {'id': 3, 'name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield': 300}, {'id': 2, 'name': 'Rice', 'moisture': 45, 'temperature': 28, 'yield': 350}]

Sorted by Yield: [{'id': 4, 'name': 'Barley', 'moisture': 25, 'temperature': 22, 'yield': 180}, {'id': 1, 'name': 'Wheat', 'moisture': 30, 'temperature': 25, 'yield': 200}, {'id': 3, 'name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield': 300}, {'id': 2, 'name': 'Rice', 'moisture': 45, 'temperature': 28, 'yield': 350}]

Search Result: {'id': 3, 'name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield': 300}

PS C:\Users\boora\Downloads\3-2\AI-AC>

Justification:

Binary Search enables quick retrieval of crop details using *crop ID. Merge Sort helps arrange crops by **soil moisture level or yield estimate*. This improves monitoring and analysis of agricultural data.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Code:

```
# Generate a Python program for an Airport Flight Management System that stores flight ID, airline name, departure time, arrival time, and status, uses Binary Search to find flights by flight ID, and uses Merge Sort to sort flights by departure time or arrival time with proper comments
```

```
# Flight records
```

```
flights = [  
    {"id":101, "airline":"IndiGo", "departure":"10:30", "arrival":"12:45", "status":"On Time"},  
    {"id":102, "airline":"Air India", "departure":"08:15", "arrival":"10:30",  
    "status":"Delayed"},  
    {"id":103, "airline":"SpiceJet", "departure":"14:00", "arrival":"16:20", "status":"On Time"},  
]
```



```
    {"id":104, "airline":"Vistara", "departure":"09:45", "arrival":"11:50", "status":"Boarding"}  
]
```

```
# Binary Search by Flight ID
```

```
def binary_search(arr, target):
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid]["id"] == target:
```

```
            return arr[mid]
```

```
        elif arr[mid]["id"] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return "Flight not found"
```

```
# Merge Sort for sorting flights
```

```
def merge_sort(arr, key):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid], key)
```

```
right = merge_sort(arr[mid:], key)
```

```
result = []
```

```
i = j = 0
```

```
while i < len(left) and j < len(right):
```

```
    if left[i][key] < right[j][key]:
```

```
        result.append(left[i])
```

```
        i += 1
```

```
    else:
```

```
        result.append(right[j])
```

```
        j += 1
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
# Sort flights by departure time
```

```
sorted_departure = merge_sort(flights, "departure")
```

```
print("Sorted by Departure Time:", sorted_departure)
```

```
# Sort flights by arrival time
```

```
sorted_arrival = merge_sort(flights, "arrival")
```

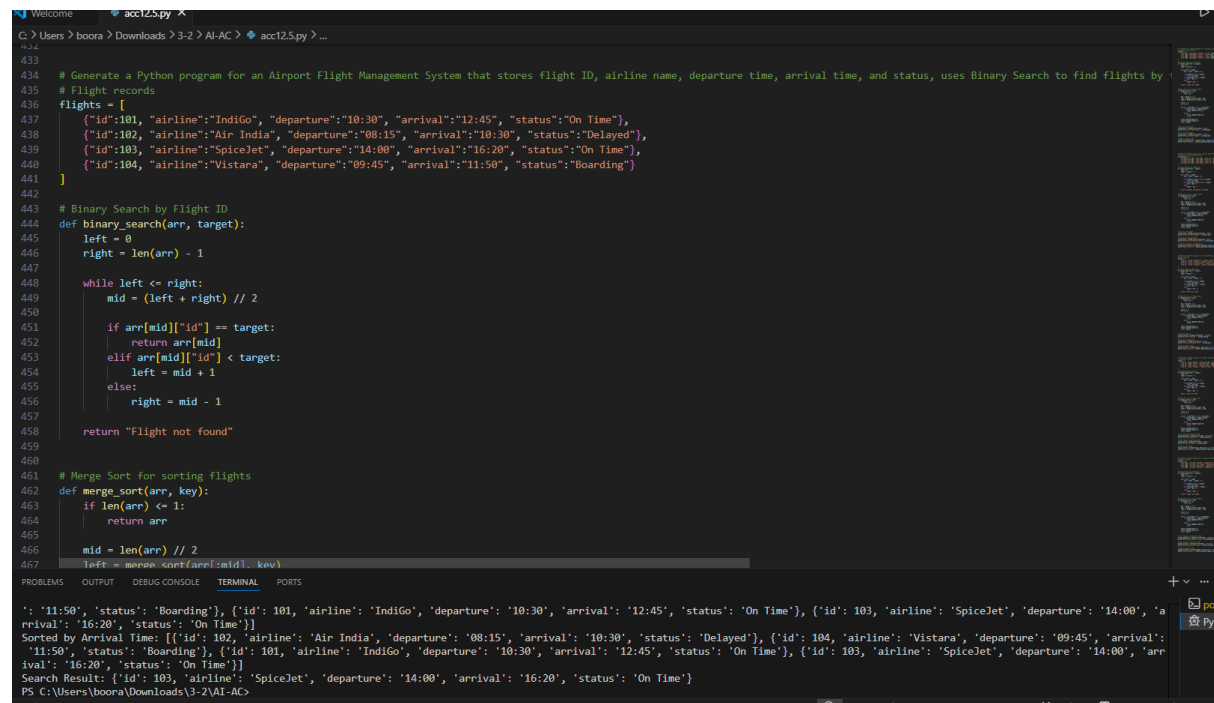
```
print("Sorted by Arrival Time:", sorted_arrival)
```

```
# Search flight by ID
```

```
flights_sorted = sorted(flights, key=lambda x: x["id"])
```

```
print("Search Result:", binary_search(flights_sorted, 103))
```

Output:



```
433
434 # Generate a Python program for an Airport Flight Management System that stores flight ID, airline name, departure time, arrival time, and status, uses Binary Search to find flights by
435 # Flight records
436 flights = [
437     {"id":101, "airline":"IndiGo", "departure":"10:30", "arrival":"12:45", "status":"On Time"},
438     {"id":102, "airline":"Air India", "departure":"08:15", "arrival":"10:30", "status":"Delayed"},
439     {"id":103, "airline":"SpiceJet", "departure":"14:00", "arrival":"16:20", "status":"On Time"},
440     {"id":104, "airline":"Vistara", "departure":"09:45", "arrival":"11:50", "status":"Boarding"}
441 ]
442
443 # Binary Search by Flight ID
444 def binary_search(arr, target):
445     left = 0
446     right = len(arr) - 1
447
448     while left <= right:
449         mid = (left + right) // 2
450
451         if arr[mid]["id"] == target:
452             return arr[mid]
453         elif arr[mid]["id"] < target:
454             left = mid + 1
455         else:
456             right = mid - 1
457
458     return "Flight not found"
459
460
461 # Merge Sort for sorting flights
462 def merge_sort(arr, key):
463     if len(arr) <= 1:
464         return arr
465
466     mid = len(arr) // 2
467     left = merge_sort(arr[:mid], key)
468     right = merge_sort(arr[mid:], key)
469
470     return merge(left, right, key)
471
472 def merge(left, right, key):
473     result = []
474     while left and right:
475         if left[0][key] < right[0][key]:
476             result.append(left.pop(0))
477         else:
478             result.append(right.pop(0))
479     result.extend(left if left else right)
480     return result
481
482 # Sort flights by arrival time
483 flights_sorted = merge_sort(flights, "arrival")
484
485 # Search for flight ID 103
486 result = binary_search(flights_sorted, 103)
487
488 # Print the result
489 print("Search Result:", result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
': '11:50', 'status': 'Boarding'}, {'id': 101, 'airline': 'IndiGo', 'departure': '10:30', 'arrival': '12:45', 'status': 'On Time'}, {'id': 103, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:20', 'status': 'On Time'}]
Sorted by Arrival Time: [{'id': 102, 'airline': 'Air India', 'departure': '08:15', 'arrival': '10:30', 'status': 'Delayed'}, {'id': 104, 'airline': 'Vistara', 'departure': '09:45', 'arrival': '11:50', 'status': 'Boarding'}, {'id': 101, 'airline': 'IndiGo', 'departure': '10:30', 'arrival': '12:45', 'status': 'On Time'}, {'id': 103, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:20', 'status': 'On Time'}]
Search Result: {'id': 103, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:20', 'status': 'On Time'}
```

Justification:

Binary Search is used to quickly find flight details using *flight ID. Merge Sort organizes flights based on **departure time or arrival time*. These algorithms help manage flight schedules efficiently.